

Module 4 – Introduction to DBMS (Lab Exercises)

1. Introduction to SQL

- 1) Create a new database named school_db and a table called students with the following columns: student_id, student_name, age, class, and address.

```
1 create database school_db;
2 use school_db;
3
4 create table students(
5     student_id int primary key auto_increment,
6     student_name varchar(255),
7     age
8     class int,
9     address varchar(255)
10 );
11
12 select *
13 from students;
```



12. Insert five records into the `students` table and retrieve all records using the `SELECT` statement.

```

1 use school_db;
2
3 insert into students (student_name, age, class, address) values
4 ('student_1', 8, 1, 'address of student_1'),
5 ('student_2', 11, 8, 'address of student_2'),
6 ('student_3', 13, 8, 'address of student_3'),
7 ('student_4', 16, 11, 'address of student_4'),
8 ('student_5', 8, 3, 'address of student_5');
9
10 select *
11 from students;

```



2. SQL Syntax

- 2.1 Write SQL queries to retrieve specific columns [student name and age] from the `students` table.

```

1 use school_db;
2
3 select student_name, age
4 from students;

```



2.2 Write SQL queries to retrieve all students whose age is greater than 10.

```
use school_db;

select *
from students
where age > 10;
```



3 SQL Constraints

- 3.1 Create a table teachers with the following columns: teacher id (Primary Key), teacher name (NOT NULL), subject (NOT NULL), and email (UNIQUE).

```
1 use school_db;
2
3 create table teachers (
4     teacher_id int primary key auto_increment,
5     teacher_name varchar(255),
6     subject varchar(255),
7     email varchar(255)
8 );
9
10 select *
11 from teachers;
```



- 3.2 Implement a FOREIGNKEY constraint to relate the teacher_id from the teachers table with the students table.

```
1 use school_db;
2
3 alter table students
4 add column teacher_id int;
5
6 alter table students
7 add constraint fk_teacher
8 foreign key (teacher_id) references teachers(teacher_id);
9
10 select *
11 from students;
```



4 MainSQLCommandsandSub-commands(DDL)

4.1 Create a table `courses` with columns: `course id`, `course name`, and `course credits`. Set the `course id` as the primary key.

```

1 use school_db;
2
3 create table courses (
4     course_id int primary key auto_increment,
5     course_name varchar(100),
6     course_credits int
7 );
8
9 select *
10 from courses;
```



4.2 Use the `CREATE` command to create a database `university_db`.

```
create database university_db;
```



5 ALTER Command

5.1 Modify the `courses` table by adding a column `course duration` using the `ALTER` command.

```

use school_db;

alter table courses
add column course_duration int;
  
```

```

1 select *
2
3 from courses;

```



5.2 Dropthe course credits column from the courses table.

```

1 use school_db;
2
3 alter table courses
4 drop column course_credits;
5
6 select *
7 from courses;

```



6 DROPCommand

6.1 Drop the teachers table from the school db database.

```
1 use school_db;
2
3 alter table students
4 drop foreign key fk_teacher;
5
6 drop table teachers;
```



6.2 Drop the student table from the school db database and verify that the table has been removed.

```
1 use school_db;
2
3 drop table students;
4
5 tables;
```




7 DataManipulationLanguage(DML)

7.1 Insert three records into the courses table using the INSERT command.

```

1 use school_db;
2
3 insert into courses (course_id, course_name, course_duration) values
4   (1, 'course1', 1),
5   (2, 'course2', 2),
6   (3, 'course3', 3);
7
8 select *
9 from courses;

```



7.2 Update the course duration of a specific course using the UPDATE command.

```

1 use school_db;
2
3 update courses
4 set course_duration = 8
5 where course_name = 'course3';
6
7 select *
8 from courses;

```



7.3 Delete a course with a specific course id from the courses table using the DELETE command.

```

1 use school_db;
2
3 delete from courses
4 where course_id = 1;
5
6 select *
7 from courses;

```



8 DataQueryLanguage(DQL)

81 Retrieve all courses from the `courses` table using the `SELECT` statement.

```

1 use school_db;
2
3 select *
4 from courses;
5

```



82 Sort the courses based on `course duration` in descending order using `ORDER BY`.

9 DataControlLanguage(DCL)

- 9.1 Create two new users user1 and user2 and grant user1 permission to SELECT from the courses table.

```
1 create user 'user1';
2
3 create user 'user2';
4
5 grant select on school_db.courses
6 to 'user1';
```



- 9.2 Revoke the INSERT permission from user1 and give it to user2.

```
1 revoke insert on school_db.courses
2 from 'user1';
3
4 grant insert on school_db.courses
5 to 'user2';
```



10 Transaction Control Language (TCL)

- 10.1 Insert a few rows into the `courses` table and use `COMMIT` to save the changes.

```
1 use school_db;
2
3 insert into courses (course_name, course_duration) values
4   (course1, 1),
5   (course2, 2),
6   (course3, 3);
7
8 commit;
9
10 select * from courses;
```



10.2 Insert additional rows, then use `ROLLBACK` undo the last insert operation.

```

1 use school_db;
2
3 start transaction;
4
5 insert into courses (course_name, course_duration) values
6     (course4, 4),
7     (course5, 2),
8     (course6, 6);
9
10 select * from courses;
11
12 rollback;
13
14 select * from courses;

```



103. Create a **SAVEPOINT** before updating the `courses` table, and use it to roll back specific changes.

```

1 use school_db;
2
3 start transaction;
4
5 delete from courses
6 where course_id=5;
7
8 select * from courses;
9
10 savepoint sp1;

```



```
22.
23. delete from courses
24. where course_id=2;
25.
26. select * from courses;
27.
28. rollback to savepoint sp1;
29.
30. select * from courses;
31.
32.
33. commit
```


11 SQL Joins

- TL1** Create two tables: departments and employees. Perform an INNER JOIN to display employees along with their respective departments.

```

1 use school_db;
2
3 create table departments (
4     department_id int primary key auto_increment,
5     department_name varchar(255)
6 );
7
8 create table employees (
9     employee_id int primary key auto_increment,
10    employee_name varchar(255),
11    employee_salary int,
12    department_id int
13 );
14
15 insert into departments (department_name) values
16     ('department1'),
17     ('department2'),
18     ('department3'),
19     ('department4');
20
21 insert into employees (employee_name, employee_salary, department_id)
22     values
23     ('employee1', 10000, 1),
24     ('employee2', 20000, 2),
25     ('employee3', 30000, 2),
26     ('employee4', 40000, 3),
27     ('employee5', 50000, 3),
28     ('employee6', 60000, 3);
29
30 select employee_id, employee_name, employee_salary, department_name
31 from employees
32 inner join departments
33 on employees.department_id = departments.department_id;

```



11.2 Use a LEFT JOIN to show all departments, even those without employees.

```

> use school_db;
>
> select departments.department_id, department_name, employee_id,
>        employee_name, employee_salary
> from departments
> left join employees
> on departments.department_id = employees.department_id;

```



12 SQL Group By

12.1 Group employees by department and count the number of employees in each department using GROUP BY.

```

use school_db;

select count(employee_id), department_id
from employees
group by department_id;

```



12.2 Use the AVG aggregate function to find the average salary of employees in each department.

```

1 use school_db;
2
3 select avg(employee_salary), department_id
4 from employees
5 group by department_id;

```



13 SQL Stored Procedure

13.1 Write a stored procedure to retrieve all employees from the employees table based on department.

```

1 use school_db;
2
3 delimiter $$
4
5 create procedure employees_department(in dep_id int)
6 begin
7     select employee_id, employee_name, employee_salary,
8           department_id
9     from employees
10    where department_id = dep_id;
11 end $$
12
13 delimiter ;
14
15 call employees_department(1);

```



13.2 Write a stored procedure that accepts `course id` as input and returns the course details.

```

1 use school_db;
2
3 delimiter $$
4
5 create procedure course_finder (in course_id int)
6 begin
7     select course_id, course_name, course_duration
8     from courses
9     where course_id = course_id;
10 end $$
11
12 delimiter ;
13
14 call course_finder(3);

```



14 SQL View

- 14.1 Create a view to show all employees along with their department names.

```

1 use school_db;
2
3 create view employees_with_departments
4 select employee_id, employee_name, employee_salary, department_name
5 from employees
6 inner join departments
7 on employees.department_id = departments.department_id;
8
9
10 select * from employees_with_departments;

```




142 Modify the view to exclude employees whose salaries are below \$80,000.

```

1 use school_db;
2
3 alter view employees_with_departments
4 select employee_id, employee_name, employee_salary, department_name
5 from employees
6 inner join departments
7 on employees.department_id = departments.department_id
8 where employee_salary >= 80000;
9
10
11 select * from employees_with_departments;
```



15 SQL Triggers

- 15.1 Create a trigger to automatically log changes to the `employees` table when a new employee is added.

```

-- use school_db;
--
-- create table employee_log (
--     log_id log_name
--     log_salary varchar(255),
--     log_dep_id int,
--     action_time int,
--     action_performestamp
--     id varchar(255)
-- );
--
-- select * from employee_log;
--
-- delimiter $$
-- create trigger log_insert
-- after insert on employees
-- for each row
-- begin
--     insert into log (log_id, log_name, log_salary, log_dep_id,
--         action_performed)
--     values (new.employee_id, new.employee_name, new.employee_salary,
--         new.department_id, 'insert');
-- end $$
-- delimiter ;

```



15.2 Create a trigger to update the last modified timestamp whenever an employee record is updated.

```

1 use school_db;
2
3 delimiter $$
4 create trigger log_update
5 before update on employees
6 for each row
7 begin
8     insert into employees_log(log_id, log_name, log_salary,
9                               log_dep_id, action_performed)
10    values (old.employee_id, old.employee_name, old.employee_salary,
11          old.department_id, 'update');
12 end; $$
13 delimiter ;

```



16 Introduction to PL/SQL

16.1 Write a PL/SQL block to print the total number of employees from the employees table.



4/4_16_1.png

16.2 Create a PL/SQL block that calculates the total sales from an orders table.



4/4_16_2.png

17 PL/SQL Control Structures

17.1 Write a PL/SQL block using an IF-THEN condition to check the department of an employee.



4/4_17_1.png

17.2 Use a FOR LOOP to iterate through employee records and display their names.



4/4_17_2.png

18 SQL Cursors

18.1 Write a PL/SQL block using an explicit cursor to retrieve and display employee details.



4/4_18.png

- 18.2 Create a cursor to retrieve all courses and display them one by one.
-



4/4_18_2.png

19 Rollback and Commit Savepoint

- 19.1 Perform a transaction where you create a savepoint, insert records, then rollback to the savepoint?

```
> use school_db;
>
> start transaction;
>
> savepoint sav_pt;
>
> insert into courses (course_name, course_duration) values
> ('course4', 4);
> ('course5', 6);
>
```

```

22 select * from courses;
23
24 rollback to savepoint sav_pt;
25
26 select * from courses;
27
28 commit

```



19.2 Commit part of a transaction after using a savepoint and then rollback the remaining changes

```
1 use school_db;
2
3 start transaction;
4
5 update courses
6 set course_name = "course4"
7 where course_id = 8;
8
9 select * from courses;
10
11 savepoint sav_pt2;
12
13 update courses
14 set course_duration = 4
15 where course_id = 8;
16
17 select * from courses;
18
19 rollback to sav_pt2;
20
21 select * from courses;
22
23 commit;
```

